

OSSP PRACTICAL QUESTIONS

NAME : B . JASMITHA

SECTION : 9

Q1 . Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

CODE :

```
GNU nano 8.7.1                                w1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];
    int pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
    }
    else if (pid == 0)
    {
        // Child process
        printf("Child Process PID: %d\n", getpid());
        printf("Executing command: %s\n", command);

        execlp(command, command, NULL);

        printf("Execution failed\n");
        exit(1);
    }
    else
    {
        // Parent process
        printf("Parent Process PID: %d\n", getpid());
        printf("Parent is waiting for child...\n");

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

LINUX COMMANDS :

```
princy@SAMUELLLL:~$ gcc w1.c
princy@SAMUELLLL:~$ ./a.out
Enter a Linux command: uname
Parent Process PID: 1827
Parent is waiting for child...
Child Process PID: 1828
Executing command: uname
Linux
Child process completed.
```

```
princy@SAMUELLLL:~$
princy@SAMUELLLL:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:         46 bits physical, 48 bits virtual
Byte Order:            Little Endian
CPU(s):                18
On-line CPU(s) list:   0-17
Vendor ID:              GenuineIntel
Model name:             Intel(R) Core(TM) Ultra 5 125H
CPU family:             6
Model:                 170
Thread(s) per core:    2
Core(s) per socket:    9
Socket(s):              1
Stepping:               4
BogoMIPS:               5990.40
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr ss
e sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good noopl xtopology tsc_reliable
nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx sse3 fma cx16 pcid sse4_1 sse4_2 x2api
c movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowpre
fetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi
1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv
1 xsave avx_vnni vnni umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_cl
ear serialize ibt flush_lld arch_capabilities

Virtualization features:
Virtualization:         VT-x
Hypervisor vendor:      Microsoft
Virtualization type:    full
Caches (sum of all):
L1d:                    432 KiB (9 instances)
L1i:                    576 KiB (9 instances)
L2:                     18 MiB (9 instances)
L3:                     18 MiB (1 instance)
NUMA:
NUMA node(s):           1
NUMA node0 CPU(s):      0-17
Vulnerabilities:
Gather data sampling:    Not affected
Ghostwrite:              Not affected
Indirect target selection: Not affected
Itlb multihit:           Not affected
L1tf:                    Not affected
Mds:                     Not affected
Meltdown:                 Not affected
Mmio stale data:         Not affected
Old microcode:           Not affected
Reg file data sampling:  Not affected
Retbleed:                 Mitigation; Enhanced IBRS
Spec rstack overflow:    Not affected
Spec store bypass:       Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:               Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:               Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRASB-eIBRS SW sequence; BHI BMI_D
IS_S
Srbds:                   Not affected
Tsa:                     Not affected
Tsx async abort:         Not affected
Vmscape:                 Not affected

princy@SAMUELLLL:~$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 356.9M 1 disk
sdb   8:16   0 159.4M 1 disk
sdc   8:32   0    2G   0 disk [SWAP]
sdd   8:48   0    1T   0 disk /mnt/wslg/distro
/
```

```
princy@SAMUELLLL:~$ ps
  PID TTY          TIME CMD
   336 pts/0    00:00:00 bash
  1866 pts/0    00:00:00 ps
princy@SAMUELLLL:~$ top
top - 06:53:38 up 15 min,  1 user,  load average: 0.08, 0.05, 0.01
top - 06:53:38 up 15 min,  1 user,  load average: 0.08, 0.05, 0.01
Tasks: 29 total,   1 running, 28 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  7651.2 total,  6980.5 free,   613.9 used,   217.9 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used,  7037.2 avail Mem
top - 06:53:51 up 15 min,  1 user,  load average: 0.14, 0.06, 0.01
Tasks: 29 total,   1 running, 28 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni,100.0 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem :  7651.2 total,  6980.6 free,   613.8 used,   217.9 buff/cache
MiB Swap:  2048.0 total,  2048.0 free,    0.0 used,  7037.4 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	23928	14280	10500	S	0.0	0.2	0:01.09	systemd
2	root	20	0	3180	2204	2072	S	0.0	0.0	0:00.02	init-systemd(Ub
6	root	20	0	3180	2040	1940	S	0.0	0.0	0:00.00	init
46	root	19	-1	50400	16884	15568	S	0.0	0.2	0:00.23	systemd-journal
81	systemd+	20	0	22540	14592	11968	S	0.0	0.2	0:00.09	systemd-resolve
93	root	20	0	35500	12416	9184	S	0.0	0.2	0:00.51	systemd-udev
110	root	20	0	2888	1944	1820	S	0.0	0.0	0:00.04	chronyd-starter
111	root	20	0	4472	3132	2884	S	0.0	0.0	0:00.02	cron
112	message+	20	0	8816	5540	4780	S	0.0	0.1	0:00.12	dbus-daemon
140	root	20	0	44092	29712	14568	S	0.0	0.4	0:00.28	networkd-dispat
172	root	20	0	18552	9504	8200	S	0.0	0.1	0:00.12	systemd-logind
206	syslog	20	0	220548	5504	4564	S	0.0	0.1	0:00.09	rsyslogd
213	root	20	0	5492	2708	2524	S	0.0	0.0	0:00.00	agetty
244	_chrony	20	0	24392	11840	7132	S	0.0	0.2	0:00.10	chronyd
245	root	20	0	123456	32688	18080	S	0.0	0.4	0:00.19	unattended-upgr
264	_chrony	20	0	12092	2428	1788	S	0.0	0.0	0:00.01	chronyd
334	root	20	0	3184	1108	980	S	0.0	0.0	0:00.00	SessionLeader
335	root	20	0	3200	1248	1108	S	0.0	0.0	0:00.33	Relay(336)
336	princy	20	0	6268	5604	3928	S	0.0	0.1	0:00.08	bash
337	root	20	0	8648	5560	4716	S	0.0	0.1	0:00.01	login
372	princy	20	0	21924	11756	9544	S	0.0	0.2	0:00.18	systemd
375	princy	20	0	22912	4104	2100	S	0.0	0.1	0:00.00	(sd-pam)
412	princy	20	0	6136	5484	3928	S	0.0	0.1	0:00.03	bash
603	polkitd	20	0	306444	8520	7704	S	0.0	0.1	0:00.06	polkitd
1876	princy	20	0	8548	5552	2980	R	0.0	0.1	0:00.29	top
1908	root	20	0	3184	1116	980	S	0.0	0.0	0:00.00	SessionLeader
1910	root	20	0	3200	1256	1108	S	0.0	0.0	0:00.00	Relay(1916)
1916	princy	20	0	6268	4404	2728	S	0.0	0.1	0:00.04	bash
1942	princy	20	0	6332	5380	2628	S	0.0	0.1	0:00.04	nano

CPU

Linux manages CPU resources using **processes and CPU scheduling**. Multiple processes can run on the same CPU because the OS schedules them and gives them CPU time.

lscpu shows the available CPU hardware, while ps and top show processes using system resources.

Memory

Linux provides programs with a **virtual memory environment** instead of requiring them to directly manage physical RAM. This allows processes to use memory while the OS manages the underlying physical resources.

Storage

Linux abstracts physical storage devices through **block devices, partitions, file systems, files and directories**. The `lsblk` command allows us to view the storage devices available to the system.

I/O

Linux provides a common interface between applications and hardware through the **kernel and device drivers**. Applications can interact with devices without needing to know the detailed hardware implementation.

Q2 . Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

CODE :

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main()
{
    int source, destination;
    char buffer[100];
    int n;

    source = open("source.txt", O_RDONLY);

    if (source < 0)
    {
        printf("Error opening source file\n");
        return 1;
    }

    destination = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0)
    {
        printf("Error opening destination file\n");
        close(source);
        return 1;
    }

    while ((n = read(source, buffer, 100)) > 0)
    {
        write(destination, buffer, n);
    }

    close(source);
    close(destination);

    printf("File copied successfully\n");

    return 0;
}
```

Q3 . Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the

CODE :

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pid;

    printf("Parent process started\n");
    printf("Parent PID: %d\n", getpid());
    printf("Parent PPID: %d\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    else if (pid == 0)
    {
        // Child process
        printf("\nChild process started\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());
        printf("Child state: Running\n");

        sleep(10);

        printf("Child state: Running again\n");
        return 0;
    }

    else
    {
        // Parent process
        printf("\nParent process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent state: Waiting for child\n");

        wait(NULL);

        printf("Child has terminated\n");
        printf("Parent state: Running\n");
    }

    return 0;
}
```

```
princy@SAMUELLLL:~$ nano w3.c
princy@SAMUELLLL:~$ gcc w3.c
princy@SAMUELLLL:~$ ./a.out
Parent process started
Parent PID: 770
Parent PPID: 401

Parent process
Parent PID: 770
Child PID: 771
Parent state: Waiting for child

Child process started
Child PID: 771
Child PPID: 770
Child state: Running

Child state: Running again
Child has terminated
Parent state: Running
```

Q4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pid1, pid2, pid3;

    pid1 = fork();

    if (pid1 == 0)
    {
        printf("Child 1: PID = %d\n", getpid());
        sleep(2);
        printf("Child 1 completed\n");
        return 0;
    }

    pid2 = fork();

    if (pid2 == 0)
    {
        printf("Child 2: PID = %d\n", getpid());
        sleep(4);
        printf("Child 2 completed\n");
        return 0;
    }

    pid3 = fork();

    if (pid3 == 0)
    {
        printf("Child 3: PID = %d\n", getpid());
        sleep(6);
        printf("Child 3 completed\n");
        return 0;
    }

    printf("Parent PID = %d\n", getpid());

    printf("Parent waiting for Child 1 using wait()\n");
    wait(NULL);

    printf("Parent waiting for Child 2 using waitpid()\n");
    waitpid(pid2, NULL, 0);

    printf("Parent waiting for Child 3 using waitpid()\n");
    waitpid(pid3, NULL, 0);

    printf("All child processes completed\n");

    return 0;
}

```

CODE :

```

princy@SAMUELLLLL:~$ nano w4.c
princy@SAMUELLLLL:~$ gcc w4.c
princy@SAMUELLLLL:~$ ./a.out
Child 1: PID = 823
Parent PID = 822
Parent waiting for Child 1 using wait()
Child 2: PID = 824
Child 3: PID = 825
Child 1 completed
Parent waiting for Child 2 using waitpid()
Child 2 completed
Parent waiting for Child 3 using waitpid()
Child 3 completed
All child processes completed

```

wait() makes the parent wait for a **child process** to finish.

The parent doesn't specify which particular child it wants to wait for.

waitpid() allows the parent to specify **which child process** it wants to wait for.

```
GNU nano 0.7.1 zombie.c
#include <stdio.h>
#include <unistd.h>

int main()
{
    int pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child process running\n");
        printf("Child PID = %d\n", getpid());
        printf("Child terminating...\n");
        return 0;
    }
    else
    {
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping...\n");

        sleep(20);

        printf("Parent completed\n");
    }

    return 0;
}
```

```
princy@SAMUELLLLL:~$ nano zombie.c
princy@SAMUELLLLL:~$ gcc zombie.c
princy@SAMUELLLLL:~$ ./a.out
Parent PID = 977
Child PID = 978
Parent sleeping...
Child process running
Child PID = 978
Child terminating...
Parent completed
princy@SAMUELLLLL:~$ |
```